Full Stack Technical Test Web Engineering

Build a *small web* — the sites, and the browser you read them in.

A few hundred one-page websites, written by strangers, living in a database. Anybody can publish another one. You are building the thing people look at them with: an address bar, links, back, forward, history, search, and a record of everywhere you have been. Everyone knows how a browser behaves. Almost nobody has built one that behaves correctly.

Stack · Next.js · NestJS · MongoDB · TypeScript Time · ~8 focused hours, 3 calendar days Deliverables · source · seed · narrated walkthrough

01

What you're building

The brief

One application, two halves that have to agree with each other. The web is the content in your database. The browser is your product. Four things it must be able to do:

01

Browse

Type an address, or follow a link, and read the page that lives at it.

02

Retrace

Back, forward, and a history anybody can scroll through and jump into.

03

Search

Find pages across the whole web by what is written in them, not just their titles.

04

Publish

Anyone can add a site: an address and a page of HTML they wrote themselves.

Everything else is yours. What the chrome around the page looks like, how addresses are shaped, how the pages are stored and rendered, where search is computed, what the API looks like, what renders on the server — none of that is specified here.

Scope discipline

This is a website, not a desktop application — it runs in a browser tab, in React on the stack above. No Electron, no native shell, no packaging. And no authentication, no accounts, no roles, no bookmarks, no tabs, no extensions, no downloads, no images or assets to host, no real network. Whoever is browsing is identified by picking a name from a list, and a publisher the same way — that name is who the history belongs to. We would rather see back, forward and history provably right than a browser with a toolbar full of buttons that half work.

02

Domain

The things in it

Site

A single page of HTML someone wrote, published at an address of its own — tidepool.zz, not "a page about tides". It has a title, a body, and an author.

Address

How a page is asked for and how it is linked to. It is the only thing typed into the address bar, and the only thing a site author can point at.

Link

A pointer from inside one page to an address. It may lead somewhere real, or nowhere that ever existed.

Visit

A recorded fact: this person arrived at this address at this time, and got there this way. Visits are what the history is made of.

That is the vocabulary, not the schema. Whether a site's HTML lives with the site or apart from it, whether an edit replaces a page or adds to it, whether history is derived from visits or kept beside them — your call.

01

Typed

An address, entered

02

Loading

Asked for, not shown

03

Shown

Read, and recorded

04

In history

Been here before

05

Nowhere

No such address

Indicative, not prescriptive. If your model has fewer states, or none at all because state is derived, say why when you walk us through it.

03

Constraints

The rules you have to work inside

The browser has to have these, and each one has to be right. What "right" means is what you already know from using a browser every day — we are not going to explain back to you.

Address bar	Type an address, go to it.
Links	Follow them, including the broken ones.
Back and forward	Both, and forward can be lost.
Restore on return	A page gone back to is the page that was left.
History	Scrollable, per person, jumpable.
Search	Across every page, by what is written in it.
Publish	An address, and a page of HTML.

Back and forward, restore on return, history, and containing HTML you did not write are not negotiable. Everything the system does has to hold them.

On the pages you are displaying

You are rendering markup you did not write into the same screen as your own interface. Anyone can publish, so assume someone will try to escape the page they published on. "Nobody would put that in a site" is not a design — be able to point at the line that contains it, and to say what it costs you in what authors can still write.

04

Data

What has to be in there before we look

Ship a seed command that gives the web something to be. Enough that browsing it feels like browsing, search has to actually choose, and history has somewhere to go.

- Roughly **10 sites** with real prose on them, linking to each other enough that you can wander several steps without turning back — including links to addresses that never existed.
- A handful of **people** to browse as, and a couple of authors with more than one site to their name.
- **An hour** of browsing history across those people: repeat visits, a long trail or two, and at least one who has been nearly everywhere.

Repeatable

Seeding twice must not double the web, and the data should be deterministic — the same command gives the same sites at the same addresses, so the walk in your recording can be walked again.

05
Submission

What to send

Deliverables

- A Git repository with real commit history — not one "initial commit" dump.
- A **narrated screen recording**, four or five minutes: a trail four deep, back and forward across it, a search and a return to its results, a publish, and an address that leads nowhere — with your voice over it explaining what you built

and why you built it that way. Tell us the model you chose, how back and forward are made to hold, how you contained the HTML you did not write, and what you knowingly left out. A silent capture is half a submission.

- Whatever it takes to **run it** — the commands for both sides and for seeding, wherever you want to put them. No write-up, just the commands.

Note on the scenario

The small web, the sites and the people in it are invented for this test. Nothing here is drawn from a live system, and your submission stays your own work — we ask only that you don't publish it as a solved interview exercise while we're still using it.

Questions are welcome. If something here is ambiguous, ask — and if you'd rather make a judgement call and tell us about it in the walkthrough, that's an equally good answer. Deciding under ambiguity is part of the job.

Full Stack Technical Test · Web Engineering